

4 Pillers of OOP

1.Encapsulation

Encapsulation means **bundling data (variables) and methods (functions) together inside a class**, and restricting direct access to some of the object's components.

Hide internal details and expose only what is necessary.

Why It Matters (Backend Thinking)

- Prevents accidental data manipulation
- Controls how data changes
- Makes systems predictable
- Protects invariants (important in finance/trading apps 🧐)

Real World Example

Think of an ATM:

- You can deposit/withdraw
- You cannot directly change your balance variable

◆ JavaScript Example

```
class BankAccount {  
  #balance; // private field  
  
  constructor(initialBalance) {  
    this.#balance = initialBalance;  
  }  
  
  deposit(amount) {  
    if (amount > 0) {
```

```
        this.#balance += amount;
    }
}

getBalance() {
    return this.#balance;
}
}

const account = new BankAccount(1000);
account.deposit(500);
console.log(account.getBalance()); // 1500
```

Notice:

- #balance is private
- Direct modification is impossible

Senior-Level Insight

Encapsulation is not about syntax (private, #).
It's about **protecting business rules**.

If your trading app allows:

```
account.balance = -1000000;
```

You have failed at encapsulation.

Abstraction

Abstraction means **hiding complex implementation details and showing only essential features**.

Focus on "what it does", not "how it does it".

Real World Example

When you call:

```
fetch("https://api.com")
```

You don't know:

- TCP handshake
- DNS resolution
- HTTP internals

That's abstraction.

```
class PaymentService {  
  processPayment(amount) {  
    this.#connectToBank();  
    this.#verifyFunds();  
    this.#transferMoney(amount);  
  }  
}
```

```
#connectToBank() {}  
#verifyFunds() {}  
#transferMoney(amount) {}  
}
```

User only calls:

```
payment.processPayment(500);
```

They don't see internal complexity.

Backend Perspective

Abstraction is:

- Controllers calling services
- Services calling repositories
- Repositories talking to DB

Each layer hides its internal logic.

That's clean architecture.

Inheritance

◆ Definition

Inheritance allows a class to **reuse properties and methods of another class**.

Child class extends parent class.

```
class Vehicle {  
  start() {  
    console.log("Vehicle started");  
  }  
}
```

```
class Car extends Vehicle {  
  drive() {  
    console.log("Car is driving");  
  }  
}
```

```
const myCar = new Car();  
myCar.start(); // inherited  
myCar.drive();
```

Advanced Insight (Important)

Inheritance is often **overused**.

Modern backend systems prefer:

Composition over inheritance

Polymorphism

◆ Definition

Polymorphism means:

Same interface, different behavior.

Real World Example

Button:

- On mobile → touch event
- On desktop → click event

Same action → different implementation.

```
class Shape {  
  draw() {  
    console.log("Drawing shape");  
  }  
}
```

```
class Circle extends Shape {  
  draw() {  
    console.log("Drawing circle");  
  }  
}
```

```
class Square extends Shape {  
  draw() {  
    console.log("Drawing square");  
  }  
}
```

```
function renderShape(shape) {  
  shape.draw();  
}
```

```
renderShape(new Circle());  
renderShape(new Square());
```

Same method → different behavior.

Deep Engineering Insight

All 4 pillars together help you build:

- Maintainable code
- Scalable systems
- Replaceable components
- Testable modules

In real production:

Pillar	Where You Use It in MERN
Encapsulation	Business logic in services
Abstraction	API layer over DB
Inheritance	Base models / shared utilities
Polymorphism	Payment methods, strategies

Q. How is encapsulation achieved in JavaScript?

- Using classes
- Using closures
- Using private fields (#)
- Using getters/setters

Q. Why is encapsulation important?

- Protects data integrity
- Prevents unauthorized access
- Enforces business rules
- Improves maintainability



Interview Tip:

Say **“Encapsulation protects invariants”** — that sounds senior-level.

Q. What is the difference between encapsulation and abstraction?

Encapsulation = Hiding data

Abstraction = Hiding implementation complexity

Q. How do you achieve abstraction in JavaScript?

- Classes
- Functions
- Modules
- Interfaces (via patterns)
- Service layers

Q. Real-world example of abstraction?

- Car steering wheel
- ATM machine
- fetch() API
- Express middleware system

Q.Types of polymorphism?

1. Compile-time (Method Overloading)

2. Runtime (Method Overriding)

Note: JavaScript supports runtime polymorphism but not true method overloading.

Feature	Method Overloading	Method Overriding
Definition	Same method name, different parameters	Same method name, redefined in child class
Happens in	Same class	Parent → Child class
Polymorphism type	Compile-time	Runtime
JS Support	 Not true overloading	 Fully supported
Requires inheritance?	No	Yes